DS 642: Applications of Parallel Computing

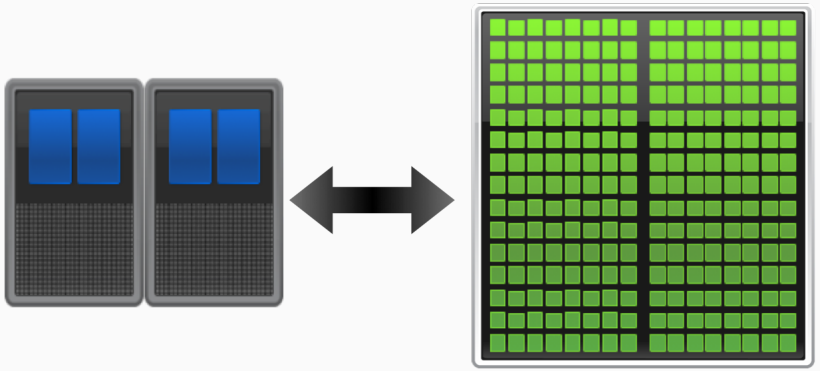
Shahriar Afkhami

Week 6-7:
Introduction to CUDA/OpenCL and Graphics Processors

What is GPU?

- Programming with GPU follows vector processors in that all processors do the same task simultaneously. In that sense, they offer substantial speed-up over sequential programs, or can over-perform programs written for distributed memory machines.
- Before GPU was introduced, CPU did all the graphic processing tasks.
- In the early 1990s, computer manufactures began to include GPU into computer systems with the aim of accelerating common graphics routines.
- GPU manufactures, such as NVIDIA and ATI/AMD, found a way to use GPUs for more general purposes, not just for graphics or videos.

What is the difference between GPU and CPU?



Latency Optimized CPU

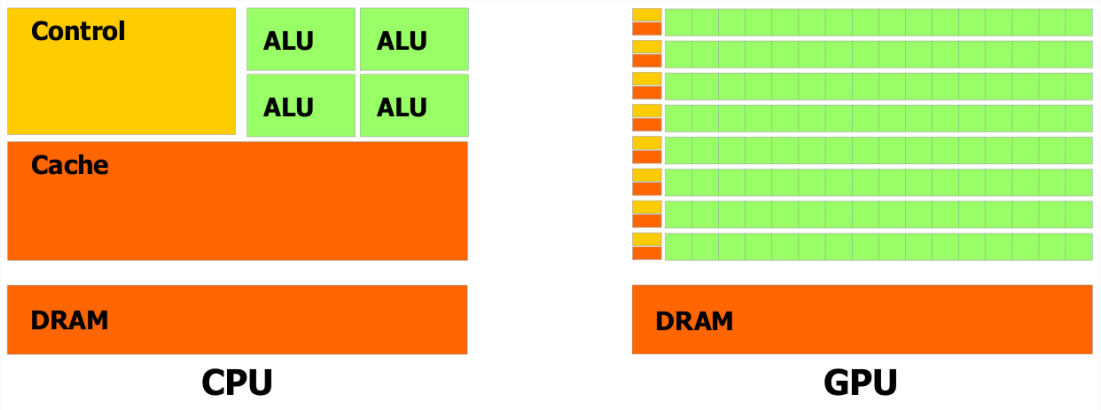
Fast Serial Processing

Throughput Optimized GPU

Scalable Parallel Processing

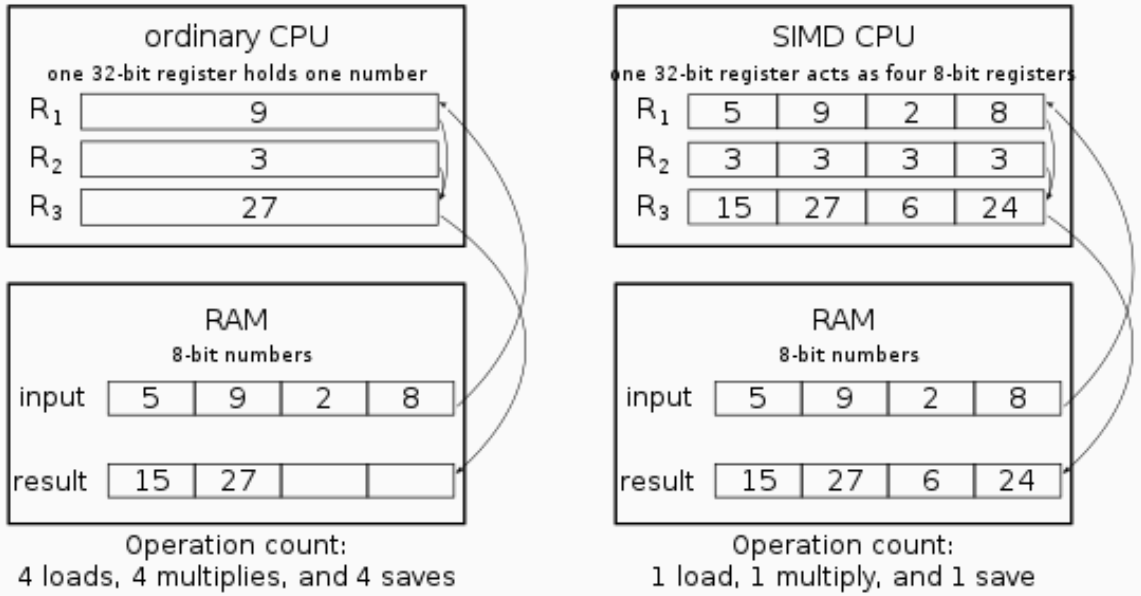
- The major difference between GPU and CPU is that GPU has a highly parallel structure.
- GPU's are more effective than CPU's if used on data that can be partitioned and processed in parallel.
- GPU is highly optimized to perform advanced calculations such as floating-point arithmetic, matrix arithmetic and so on.

Heterogeneous Parallel Computing



- Different goals produce different designs
  - Manycore assumes work load is highly parallel
  - Multicore must be good at everything, parallel or not
- Multicore: minimize latency experienced by 1 thread
  - lots of big on-chip caches
  - extremely sophisticated control
- Manycore: maximize throughput of all threads
  - lots of big ALUs
  - multithreading can hide latency ... so skip the big caches
  - simpler control, cost amortized over ALUs via SIMD

SIMD



- Single Instruction Multiple Data architectures make use of data parallelism
- Amortize control overhead over SIMD width
  - OpenMP / Pthreads / MPI all neglect SIMD parallelism
  - Because it is difficult for a compiler to exploit SIMD
  - Many languages (like C) are difficult to vectorize

The CUDA Programming Model

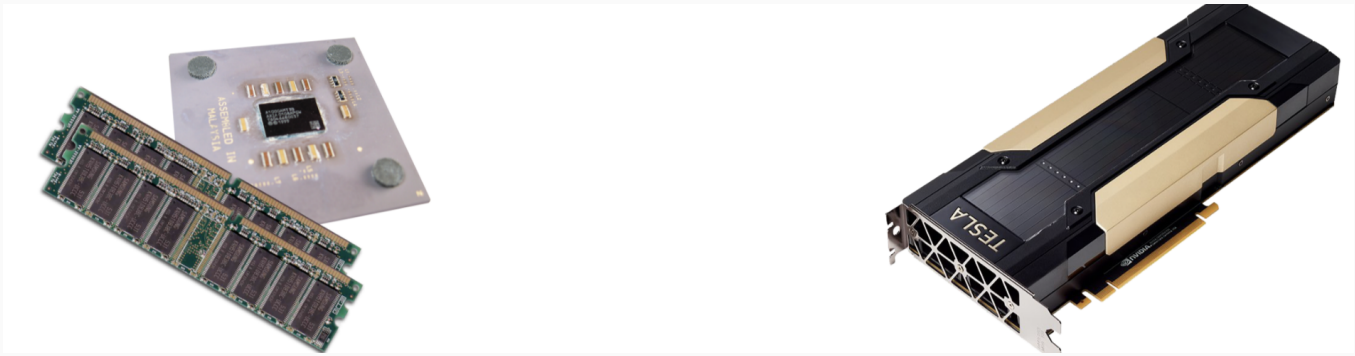
- CUDA is a programming model designed for:
  - Manycore architectures
  - WideSIMDparallelism
  - Scalability
- CUDA provides:
  - A thread abstraction to deal with SIMD
  - Synchronization & data sharing between small groups of threads
- CUDA programs are written in C++ with minimal extensions: set of extensions to enable heterogeneous programming
- OpenCL is inspired by CUDA, but HW & SW vendor neutral: similar programming model, C only for device code

Simple Processing Flow

```
memory_allocation();
copy_input_from_host_to_device();
load_and_execute_gpu_code<<<nBlk, nTid>>>(args);
cudaDeviceSynchronize();
copy_result_from_device_to_host();
memory_cleanup();
```

- Copy input data from CPU memory to GPU memory
- Load GPU program and execute, caching data on chip for performance
- Copy results from GPU memory to CPU memory

Simple Processing Flow



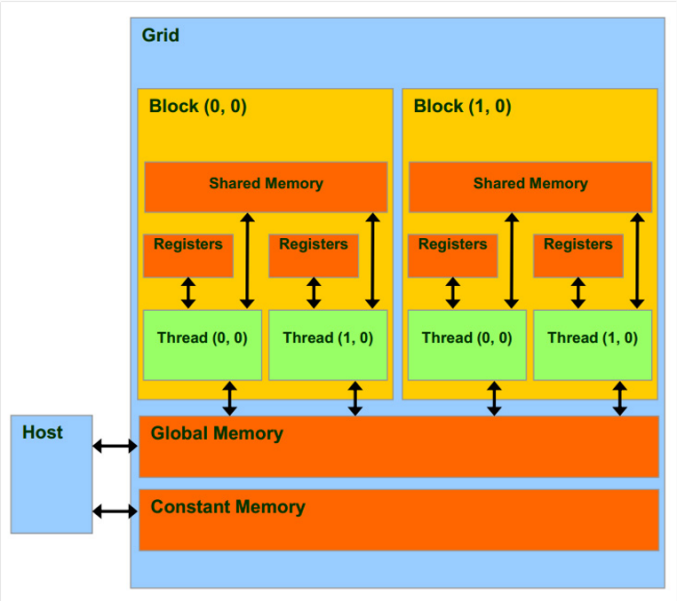
- Host: The CPU and its memory (host memory)
- Device: The GPU and its memory (device memory)

CUDA “Hello World”

```
1  #include <stdio.h>
2
3  __global__ void hello(){
4
5    printf("Hello from block: %u, thread: %u\n",  );
6  }
7
8  int main(){
9
10   hello<<<  >>>();
11   cudaDeviceSynchronize();
12 }
```

Triple angle brackets mark a call to device code, also called kernel launch.
The parameters inside the triple angle brackets are the CUDA kernel execution configuration.

Hierarchy of Grids, Blocks, and Threads



- Parallel kernels composed of many threads: all threads execute the same sequential program
- Threads are grouped into thread blocks: threads in the same block can cooperate via extremely fast on-chip (but small) shared memory
- Threads/blocks have unique IDs
- Independent blocks can form grids

CUDA Function Declarations

```
__global__
```

- Kernel function (must return void)
- Invoked from host
- Executed in parallel on device

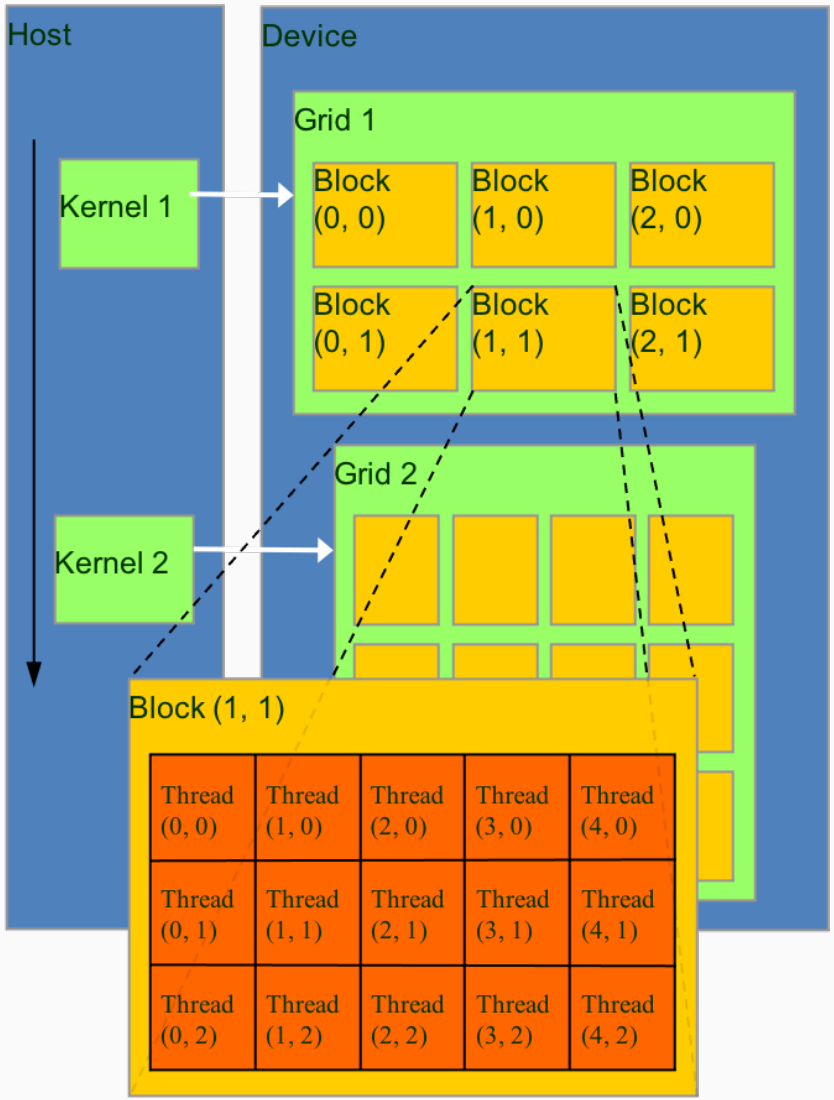
```
__host__
```

- Called and executed on host

```
__device__
```

- Called and executed on device
- Restrictions: no recursion, no static variables, etc.

GPU Programming



CUDA “Hello World”

```
1  #include <stdio.h>
2  #define NUM_BLOCKS 4
3  #define BLOCK_WIDTH 8
4
5  /* Function executed on device (GPU) */
6  __global__ void hello(void){
7
8    printf("Hello from GPU: thread %d and block %d\n",
9            threadIdx.x, blockIdx.x );
10 }
11 int main(void){
12
13 /* execute function on device (GPU) */
14   hello<<<NUM_BLOCKS, BLOCK_WIDTH>>>();
15
16 /* wait until all threads finish their job */
17   cudaDeviceSynchronize();
18 }
```

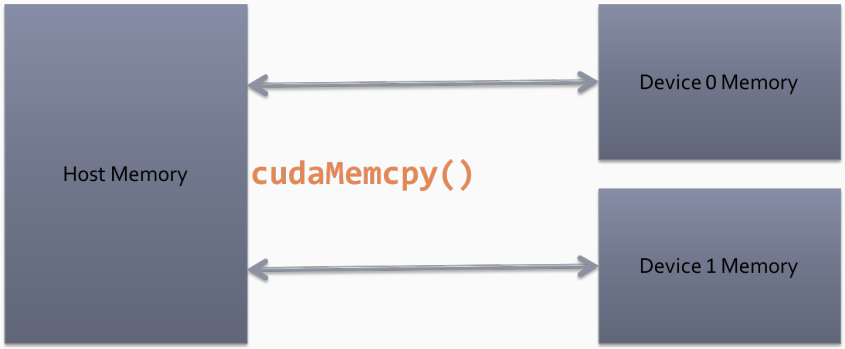
CUDA “Hello World”

Compile and build the program using NVIDIA’s `nvcc` compiler:

```
nvcc -o helloCuda helloCuda.cu
```

`nvcc` separates source code into host and device components:
Device functions (e.g. kernel()) processed by NVIDIA compiler: `nvcc`
Host functions (e.g. main()) processed by standard host compiler: `gcc`.
`nvcc` can compile both C and C++ codes, and the suffix should be `.cu`

Memory Model



Basic device memory management:

```
/* Explicit memory allocation returns pointers
   to GPU memory */
cudaMalloc()
/* Explicit memory copy for host to/from device,
   device to/from device */
cudaMemcpy()
/* Free memory */
cudaFree()
```

First CUDA code

```
1  #include <stdio.h>
2  #define NUM_BLOCKS 4
3  #define BLOCK_WIDTH 8
4
5  /* Function executed on device (GPU) */
6  __global__ void hello(void){
7
8    printf("Hello from GPU: thread %d and block %d\n",
9            threadIdx.x, blockIdx.x );
10 }
11 int main(void){
12
13 /* execute function on device (GPU) */
14   hello<<<NUM_BLOCKS, BLOCK_WIDTH>>>();
15
16 /* wait until all threads finish their job */
17   cudaDeviceSynchronize();
18 }
```

Second CUDA code

A simple program that uses a GPU kernel function to add two numbers on the GPU, and then brings the results from the GPU back into the CPU for output. The Host Code:

```
1  int main(int argc, char* argv[]) {
2
3  // Read input values
4    int a = atoi(argv[1]);
5    int b = atoi(argv[2]);
6
7  // c is storage place for main memory result
8    int c;
9
10 // dev_c is storage place for result on GPU (device)
11   int *dev_c;
12
13 // Allocate memory on GPU to store result
14   cudaMalloc((void**)&dev_c, sizeof(int));
```

Second CUDA code

```
1  // Use one GPU unit to perform addition and store result
2  // in dev_c
3    add<<<1,1>>>(a, b, dev_c);
4
5  // Move result into main memory from GPU
6    cudaMemcpy(&c, dev_c, sizeof(int), cudaMemcpyDeviceToHost);
7
8    printf("%d + %d = %d\n", a, b, c);
9
10 // Free the allocated memory on GPU.
11   cudaFree(dev_c);
12
13   return 0;
14 }
```

Second CUDA code

```
1  // Use one GPU unit to perform addition and store result
2  // in dev_c
3    add<<<1,1>>>(a, b, dev_c);
4
5  // Move result into main memory from GPU
6    cudaMemcpy(&c, dev_c, sizeof(int), cudaMemcpyDeviceToHost);
7
8    printf("%d + %d = %d\n", a, b, c);
9
10 // Free the allocated memory on GPU.
11   cudaFree(dev_c);
12
13   return 0;
14 }
```

Second CUDA code

Device code:

```
1  // Output is a pointer to address where result will be stored.
2  // The address is assumed to be a GPU memory address.
3
4  __global__ void add(int a, int b, int* c) {
5    *c = a + b;
6  }
```

Measuring Performance

The tool we use to measure the time GPU spends on a task is the CUDA Event API.

```
1  // define events in the field
2  cudaEvent_t     start, stop;
3  // create two events we need
4  HANDLE_ERROR( cudaEventCreate( &start ) );
5  HANDLE_ERROR( cudaEventCreate( &stop ) );
6  // instruct the runtime to record the event start.
7  HANDLE_ERROR( cudaEventRecord( start, 0 ) );
8  // get stop time, and display the timing results
9  HANDLE_ERROR( cudaEventRecord( stop, 0 ) );
10 HANDLE_ERROR( cudaEventSynchronize( stop ) );
11 float   elapsedTime;
12 HANDLE_ERROR( cudaEventElapsedTime( &elapsedTime,
13                 start, stop ) );
14 printf( "Time to generate:  %3.1f ms\n", elapsedTime );
```

Memory allocation and copying

The command `cudaMalloc()`, similar to `malloc()` command in standard C, tells the CUDA runtime to allocate memory on the device (Memory of GPU):

```
1 // allocate memory on the GPU
2 HANDLE_ERROR( cudaMalloc( (void**)&dev_a, N * sizeof(int) ) );
```

We can use `cudaMemcpy()` from host code to place data in memory on a device, similar to `memcpy()` command in standard C:

```
1 // copy arrays 'a'
2 HANDLE_ERROR( cudaMemcpy( dev_a, a, N * sizeof(int),
3                 cudaMemcpyHostToDevice ) );
```

Or transfer data back from device (GPU) to the host (CPU)

```
1 cudaMemcpy( a, dev_a, N * sizeof(int),
2             cudaMemcpyDeviceToHost );
```

Example of vector addition

The Device Code:

```
1
2
3 __global__ void vector_add(float *out, float *a, float *b, int n) {
4     for(int i = 0; i < n; i ++){
5         out[i] = a[i] + b[i];
6     }
7 }
```

The Kernel Invocation:

```
1 vector_add<<<1,1>>>(d_out, d_a, d_b, N);
```

This vector addition in CUDA uses only one GPU thread.

Example of vector addition

The Host Code:

```
1  int main(){
2      float *a, *b, *out;
3      float *d_a, *d_b, *d_out;
4
5      // Allocate host memory
6      a   = (float*)malloc(sizeof(float) * N);
7      b   = (float*)malloc(sizeof(float) * N);
8      out = (float*)malloc(sizeof(float) * N);
9
10     // Initialize host arrays
11     for(int i = 0; i < N; i++){
12         a[i] = 1.0f;
13         b[i] = 2.0f;
14     }
15
16     // Allocate device memory
17     cudaMalloc((void**)&d_a, sizeof(float) * N);
18     cudaMalloc((void**)&d_b, sizeof(float) * N);
19     cudaMalloc((void**)&d_out, sizeof(float) * N);
20
21     // Transfer data from host to device memory
22     cudaMemcpy(d_a, a, sizeof(float) * N, cudaMemcpyHostToDevice);
23     cudaMemcpy(d_b, b, sizeof(float) * N, cudaMemcpyHostToDevice);
24
25     // Executing kernel
26     vector_add<<<1,1>>>(d_out, d_a, d_b, N);
27
```

Example of vector addition

The Host Code Continued:

```
1
2       // Transfer data back to host memory
3       cudaMemcpy(out, d_out, sizeof(float) * N, cudaMemcpyDeviceToHost);
4
5       // Verification
6       for(int i = 0; i < N; i++){
7           assert(fabs(out[i] - a[i] - b[i]) < MAX_ERR);
8       }
9       printf("out[0] = %f\n", out[0]);
10      printf("PASSED\n");
11
12      // Deallocate device memory
13      cudaFree(d_a);
14      cudaFree(d_b);
15      cudaFree(d_out);
16
17      // Deallocate host memory
18      free(a);
19      free(b);
20      free(out);
21  }
```

Compiling

On Bridges-2, GPU nodes are compute capability 70 (Volta and Tesla):

```
$ nvcc add_vector.cu -o add_vector -lm \
-arch=compute_70 -code=sm_70
```

which is a shorthand for:

```
-gencode=arch=compute_70,code=sm_70 \
-gencode=arch=compute_70,code=compute_70
```

Examples of vector addition in parallel

The Device Code:

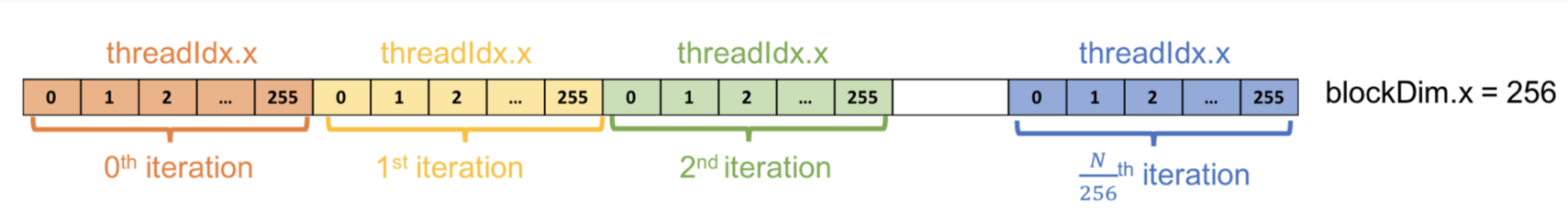
```
1
2  __global__ void vector_add_gird(float *out, float *a, float *b) {
3      out[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
4  }
5
6  __global__ void vector_add_block(float *out, float *a, float *b) {
7      out[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
8  }
9
10 __global__ void vector_add(float *out, float *a, float *b, int n) {
11     int index = threadIdx.x;
12     int stride = blockDim.x;
13
14     for(int i = index; i < n; i += stride){
15         out[i] = a[i] + b[i];
16     }
17 }
```

The Kernel Invocation:

```
1  vector_add<<<1,N>>>(d_out, d_a, d_b);
2  vector_add<<<N,1>>>(d_out, d_a, d_b);
3  vector_add<<<1,256>>>(d_out, d_a, d_b, N);
```

Examples of vector addition in parallel

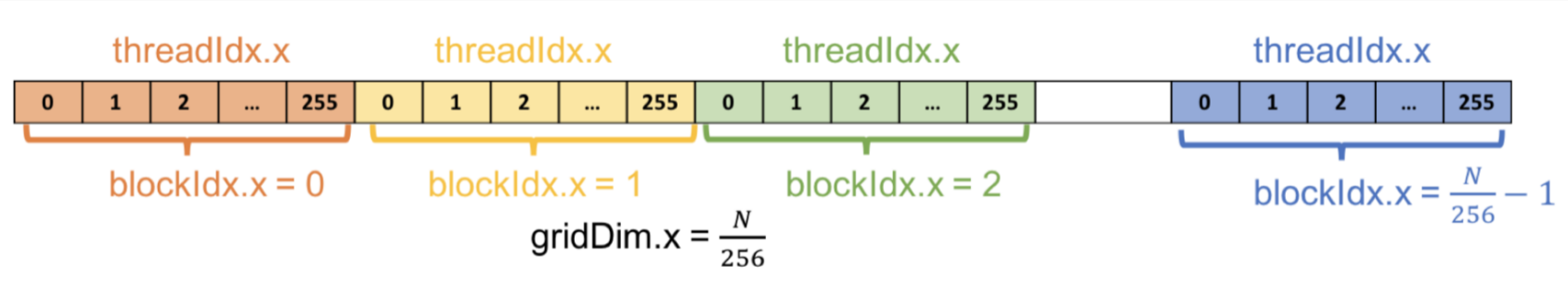
For the k-th thread, the loop starts from k-th element and iterates through the array with a loop of 256.



- `threadIdx.x` index of the thread within the block
- `blockDim.x` size of thread block (number of threads in the thread block)
- `blockIdx.x` index of the block within the grid
- `gridDim.x` size of the grid

Combining Blocks and Threads

To take advantage of CUDA GPUs, kernel should be launched with multiple thread blocks. We will use multiple thread blocks to create *N* threads; each thread processes an element of the arrays:

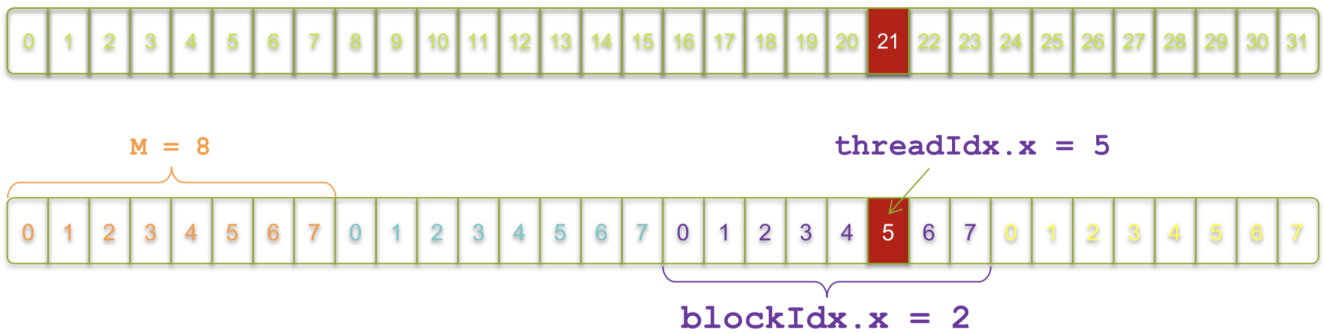


To assign a thread to a specific element, we need to know a unique index for each thread:

```
1  int index = threadIdx.x + blockIdx.x * blockDim.x;
```

Combining Blocks and Threads

```
1
2  __global__ void vector_add(float *out, float *a, float *b, int n) {
3      int index = blockIdx.x * blockDim.x + threadIdx.x;
4      out[index] = a[index] + b[index];
5
6  }
7
8  __global__ void vector_add(float *out, float *a, float *b, int n) {
9      int index = blockIdx.x * blockDim.x + threadIdx.x;
10
11     // Handling arbitrary vector size
12     if (index < n){
13         out[index] = a[index] + b[index];
14     }
15 }
```




```
int index = threadIdx.x + blockIdx.x * M;
          =      5      +     2      * 8;
          = 21;
```

Combining Blocks and Threads

```

__global__ void vector_add(float *out, float *a, float *b, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    out[index] = a[index] + b[index];

}

__global__ void vector_add(float *out, float *a, float *b, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    // Handling arbitrary vector size
    if (index < n){
        out[index] = a[index] + b[index];
    }
}
```

```
vector_add<<<N/THREADS_PER_BLOCK,THREADS_PER_BLOCK>>>(d_out, d

int block_size = 256;
int grid_size = (N + block_size) / block_size;
vector_add<<<grid_size,block_size>>>(d_out, d_a, d_b, N);
```

Profiling Performance

NVIDIA provides a command line profiler tool; which give a more insight information of CUDA program performance.

To profile our vector addition, use following command

```
1 $ nvprof ./vector_add
```

Summary

- Manycore processors provide useful parallelism
- CUDA makes parallel programming easy to use SIMD
- CUDA SIMD allows highly scalable algorithm design and implementation
- CUDA explicit parallelism:
  - Programmer's responsibility to schedule resources
  - Decompose algorithm into kernels
  - Decompose kernels into blocks
  - Decompose blocks into threads